Software: Practice and Experience

WILEY

Journal of Software: Practice and Experience

ORIGINAL ARTICLE

Proof-Carrying Source-Level Verification with Small-Kernel Certificate Checking

Zichen Song[1,2] | Weijia Li[2]

[1]Sungkyunkwan University, Republic of Korea | [2]Lanzhou University, China |

Correspondence: Correspondence: Zichen Song, Sungkyunkwan University, Republic of Korea. Email: sls530@skku.edu

Keywords: source-level verification | proof certificates | Rust verification | small-kernel checking | trusted computing base

ABSTRACT

Modern deductive verifiers for Rust programs increasingly support source-level reasoning over ownership, borrowing, mutable references, arrays, loops, and algebraic data types. However, their verification results are typically trusted through a large proof-producing implementation that combines proof search, symbolic execution, rule application, simplification, and proof-state management. This paper presents a proof-carrying source-level verification framework that separates proof production from proof trust. Instead of treating a Rust verification engine as the final trusted authority, our approach uses it as an untrusted proof producer and exports its proof evidence into replayable JSON certificates. We then design a small-kernel certificate checker that independently validates proof structure, rule schemas, branch closure, substitutions, side conditions, and replay digests under multiple replay modes, including precise replay, schema-level textual replay, conservative trace-shape replay, and conservative arithmetic replay. To improve practicality, the framework also supports compression-preserving macro certificates, enabling compact proof evidence while retaining independent checkability. Theoretically, the framework is based on the principle that an accepted certificate corresponds to a replayable derivation under the supported rule schemas and replay discipline, thereby reducing the trusted computing base from a large verifier to a compact checker and its rule interface. We implement the framework as a reproducible artifact on top of Rust source-level verification traces and evaluate it on representative Rust verification examples involving assignments, borrowing, mutable references, tuples, arrays, enums, loop invariants, arithmetic reasoning, and binary search. The results show that full and compressed certificates can be checked efficiently by a small checker, while systematically corrupted certificates are rejected with explicit diagnostics. These findings suggest that proof-carrying certificate checking is a practical and auditable way to strengthen trust in source-level Rust verification systems. The source code is available at https://github.com/shjdjjfi/Software-Practice-and-Experience-Journal-UnderReview-Paper-Zichen-Song-.git.

1 | Introduction

Deductive verification has become an important technique for improving the reliability of safety-critical and correctness-critical software. For languages such as Rust, source-level verification is particularly attractive because it can reason directly about ownership, borrowing, mutable references, arrays, tuples, loops, and other language-level constructs without translating the program into a substantially different intermediate representation. However, practical verification tools are often large and complex software systems. They typically combine parsing, symbolic execution, verification-condition generation, proof search, rule application, arithmetic simplification, and proof-state management inside a single verification engine.

This monolithic design creates a practical trust problem. When a verifier reports that a program satisfies its specification, the user must often trust the entire verification stack, including proof-search heuristics, rule implementations, simplification procedures, and internal proof-state transformations. Such a trusted boundary is difficult to audit, difficult to reproduce independently, and difficult to validate against implementation errors.

This issue is especially important for source-level Rust verification, where the verifier must account for features such as borrowing and mutation that directly affect program safety.

A natural way to reduce this trust burden is to separate proof production from proof checking. A large verifier may still be used to discover proofs, but its output should be converted into explicit proof evidence that can be checked by a small, independent, and conservative checker. This proof-carrying workflow makes the final verification result depend on replayable certificates rather than on the full internal implementation of the original verification engine. In this work, we present such a framework for certificate-carrying source-level verification, with a particular focus on small-kernel certificate checking, certificate compression, negative validation, and reproducible artifact evaluation. The uploaded project describes this design as a workflow in which a large verifier acts only as an untrusted proof producer, while a small checker makes the final accept/reject decision.

This paper addresses the following problem: how can source-level verification results be made independently checkable without requiring users to trust the entire verification engine? More concretely, given a source program, a specification, and a proof trace generated by an existing verification backend, our goal is to construct a certificate-based verification workflow satisfying three practical requirements. First, the proof evidence should be explicit and inspectable, rather than being hidden inside the internal data structures of the proof producer. Second, the certificate should be checkable by a small independent component that does not invoke the original proof engine during replay. Third, the checker should be conservative: unsupported rules, malformed proof structure, invalid side-condition evidence, corrupted substitutions, broken branch closure, or invalid compressed macro steps must lead to rejection rather than unsound acceptance.

This problem is non-trivial because proof traces emitted by practical verification tools are often heterogeneous and incomplete. Some traces contain textual sequents and detailed rule applications, while others expose only proof-tree structure, rule names, branch information, or opaque proof-state snapshots. A useful certificate checker must therefore support multiple replay levels. When sufficient evidence is available, it should perform precise rule-schema replay. When only partial evidence is available, it should clearly report conservative trace-shape replay instead of pretending to reconstruct information that is not present. In addition, certificate compression must not be treated as a trusted optimization: compressed macro steps must themselves remain checkable.

We implement this idea in a proof-carrying verification artifact for Rust-oriented source-level verification. The framework converts proof traces into JSON certificates, checks full and compressed certificates using a compact replay kernel, reports replay statistics, and evaluates robustness through negative tests. The benchmark results cover primitive assignment, borrowing, mutable references, tuples, loops, arrays, enums, arithmetic, and binary-search-style examples, with all positive examples accepted and all final open goals closed. The trusted checker reported in the benchmark is 205 lines of code, while certificate checking is substantially faster than proof production across the included examples.

This paper makes the following contributions:

- **A proof-carrying workflow for source-level Rust verification.** We present a software architecture that separates proof production from proof trust. Existing verification infrastructure is used as an untrusted proof producer, while verification results are exported as explicit JSON proof certificates. This design reduces the trusted boundary from a large verifier-centered implementation to a small certificate-checking component.
- **A small-kernel checker for replayable full and compressed certificates.** We design and implement an independent checker that validates certificate structure, rule identifiers, parent-child dependencies, branch metadata, side-condition evidence, substitutions, closure information, and replay digests. The checker supports both full certificates and compressed macro certificates, allowing proof evidence to be reduced in size while preserving independent checkability.
- **A reproducible evaluation with positive benchmarks and negative certificate validation.** We provide an artifact-level evaluation on representative Rust verification examples involving assignment, borrowing, mutable references, tuples, loop invariants, arrays, enums, arithmetic, and binary-search-style programs. The evaluation reports proof size, compressed certificate size, replay time, compression ratio, and trusted checker size. We also include negative tests that deliberately corrupt rule identifiers, substitutions, side conditions, initial sequents, branch closure evidence, and compressed macro steps; all such corrupted certificates are rejected with explicit diagnostic categories.

2 | Background and Motivation

At the same time, Rust verification introduces engineering challenges that are less visible in simpler imperative languages. A source-level proof must preserve the meaning of ownership transfer, shared and mutable borrowing, reference dereferencing, heap updates, pattern matching, and loop-carried state changes. These features often interact within the same proof obligation. For example, a loop over an array may require reasoning simultaneously about index arithmetic, borrow validity, mutable updates, and invariant preservation. Therefore, a Rust-oriented verifier needs not only a sound program logic, but also a reliable implementation strategy for recording, replaying, and validating the proof steps generated for such language-specific constructs.

2.1 | Source-Level Verification for Rust

Rust has become an important systems programming language because it provides strong ownership, borrowing, and lifetime disciplines while still allowing low-level control over memory and performance. These properties make Rust attractive for safety-critical and infrastructure software, where memory safety, aliasing control, and predictable resource management are essential. However, Rust's type system alone does not prove full functional correctness. Properties such as loop invariants, array bounds, arithmetic postconditions, data-structure consistency, and specification-level behavior still require additional reasoning beyond compilation.

Source-level deductive verification addresses this gap by reasoning directly about Rust programs and their specifications. Instead of translating Rust code into a distant intermediate language and verifying the translated artifact, a source-level verifier attempts to preserve the structure of the original program during verification. This is particularly important for Rust, because ownership, borrowing, mutable references, pattern matching, tuples, arrays, and loop invariants are not merely syntactic features; they determine the shape of the proof obligations generated during verification.

Existing source-level verification engines for Rust typically combine several tasks inside one large verification pipeline. Such a pipeline may parse source programs, generate verification conditions, apply dynamic-logic rules, perform symbolic execution, simplify arithmetic constraints, manage proof branches, and close proof goals. This integration is useful for automation and usability, but it also creates a large trusted implementation. If the verifier reports that a program satisfies its specification, the user must trust not only the logical rules, but also the implementation of proof search, proof-state management, simplification, branching, and final proof closure.

This observation motivates a different architecture for source-level verification. The verification engine should remain useful as a proof producer, but the final trust decision should not depend on the entire proof-producing system. Instead, the engine should emit explicit proof evidence that can be checked independently. This paper studies such a workflow for Rust source-level verification, where verification results are represented as proof certificates and validated by a small certificate checker.

2.2 | Verifier-Centered Trust Boundaries

A common software-verification workflow is verifier-centered: the verification engine receives a program and a specification, performs proof search or proof construction internally, and returns a final result such as `verified` or `failed`. In this workflow, the verifier itself is the trusted authority. The proof may exist internally as a proof tree, a trace, a tactic script, or a collection of proof obligations, but the final user-facing result is often accepted because it is produced by the verifier.

This design is convenient, but it places a broad implementation inside the trusted computing base. The trusted boundary may include source parsing, front-end encoding, rule application, symbolic execution, arithmetic simplification, branch scheduling, proof caching, proof compression, and the logic used to decide whether all proof obligations are closed. Any unsoundness in these components may compromise the final verification result. For an engineering-oriented verification tool, this is a practical concern: the larger the trusted boundary is, the harder it becomes to audit, test, reproduce, or independently validate the result.

The problem is especially visible in source-level verification for Rust. Rust programs involve language features whose verification requires careful handling of references, mutable updates, ownership-inspired reasoning, arrays, tuples, enums, and loops. A verifier may correctly automate many of these cases, but the implementation of the automation is necessarily complex. If the final claim of correctness depends on the entire verifier, then the user must trust a large system rather than a compact logical core. A verifier-centered boundary also makes independent reproduction difficult. A proof result may depend on a particular verifier version, proof-search heuristic, simplification strategy, or internal proof representation. Even when a proof trace is available, it may not be structured as an independently replayable certificate. As a result, the distinction between producing a proof and checking a proof becomes blurred.

Our motivation is to separate these concerns. The verifier may remain large, heuristic, and implementation-heavy, but it should be treated as an untrusted producer of candidate proof evidence. The final accept/reject decision should be made by a smaller component whose role is limited to certificate validation. This changes the trust boundary from "trust the verifier" to "trust the checker and the explicitly supported replay discipline."

2.3 | Proof-Carrying Verification

Proof-carrying verification follows the principle that a verification result should be accompanied by explicit evidence. In such a workflow, a proof producer generates a certificate, and a proof checker independently validates whether the certificate is acceptable. The producer may be complex and optimized for automation, while the checker is designed to be simpler, conservative, and auditable. In the setting of source-level Rust verification, a certificate can record the structure of the derivation produced by a verification engine. Such a certificate may include proof-step identifiers, rule names, parent-child dependencies, branch identifiers, side-condition information, substitutions, initial and final proof states, closure evidence, and replay digests. The purpose is not merely to store a log file, but to provide replayable proof evidence that can be inspected, compressed, corrupted for testing, and rechecked independently.

This separation has several practical advantages. First, it improves auditability: users can inspect the certificate rather than only trusting a binary verifier result. Second, it improves reproducibility: the same certificate can be checked again without rerunning the original proof search. Third, it improves diagnosability: malformed rules, broken substitutions, invalid branch closures, inconsistent side conditions, and corrupted macro steps can be rejected with explicit error categories. Fourth, it enables proof-certificate compression, where large proof traces can be represented by checked macro steps while still preserving replay evidence. In our framework, the proof-producing verifier is not part of the final trusted decision. It may discover the proof, but it does not decide whether the proof evidence is trustworthy. The certificate checker reads the generated certificate and validates it according to supported rule schemas, replay modes, branch structure, substitution evidence, side-condition checks, and digest consistency. Unsupported or underspecified evidence is rejected rather than silently accepted. This design is particularly suitable for software-engineering practice. A verification pipeline can still use an existing large verifier for proof automation, but the verification result becomes more portable and reviewable. The certificate acts as an explicit interface between proof production and proof trust.

2.4 | Why Small-Kernel Certificate Checking?

The central motivation for small-kernel certificate checking is to reduce the trusted computing base of source-level verification. A full verification engine may contain many thousands of

lines of implementation logic, including proof search, front-end processing, simplification, and heuristic automation. In contrast, a certificate checker can be designed around a much smaller task: validate whether a given certificate conforms to a supported replay discipline.

A small checker is easier to audit and test because its responsibilities are narrow. It does not need to search for proofs, guess invariants, perform large-scale symbolic exploration, or reimplement the full verifier. Instead, it checks certificate structure, known rule schemas, parent-child proof dependencies, branch metadata, closure evidence, substitutions, side conditions, and replay digests. If the certificate uses an unsupported rule family or omits required evidence, the checker rejects it. This conservative behavior is essential: the checker should fail closed rather than accept proof evidence that it cannot validate.

Small-kernel checking also provides a natural way to evaluate trust reduction. In our implementation, the trusted checker is intentionally compact, while the larger proof-producing system is treated as untrusted proof infrastructure. The benchmark results show that certificates can be generated and checked across examples covering primitive assignment, borrowing, mutable references, tuples, arrays, enums, arithmetic rules, loop invariants, and binary-search-style verification tasks. The checker accepts valid certificates and rejects corrupted ones, including certificates with unsupported rule identifiers, corrupted substitutions, missing side conditions, wrong initial sequents, invalid branch closure evidence, and corrupted compressed macro steps.

This architecture does not claim to replace a deductive verifier. It does not improve proof search by itself, nor does it attempt to support the full Rust language in one step. Its contribution is instead architectural and practical: it changes the final trust decision from a verifier-centered decision to a certificate-centered decision. A large verifier may still be used where it is strongest, namely proof production and automation, while a small independent checker is used where trust matters most, namely final validation. For Software: Practice and Experience, this distinction is important. The software contribution is not only a new verification artifact, but also a reusable engineering pattern for making verification results more auditable, replayable, and robust against implementation-level trust assumptions.

3 | Overview of the Proposed Framework

This section presents the proposed proof-carrying source-level verification framework. The central idea is to separate proof generation from proof trust. Instead of requiring users to trust the full verification engine, the framework treats the verifier as an untrusted proof producer and delegates the final acceptance decision to a small independent certificate checker.

3.1 | Design Goals

The framework is designed around four main goals.

First, the framework aims to reduce the trusted computing base of source-level verification. Existing deductive verifiers usually combine parsing, symbolic execution, proof search, proof-rule application, simplification, arithmetic reasoning, and proof-state management in one large implementation. Although this design is effective for automation, it makes the final verification result depend on a large and complex trusted system. In contrast, our framework keeps the original verifier outside the trusted boundary and relies on a small certificate checker for the final decision.

Second, the framework aims to make proof evidence explicit and auditable. Rather than treating a proof trace as an internal artifact of the verifier, the framework exports a structured JSON certificate. The certificate records proof steps, rule identifiers, parent-child dependencies, branch information, side-condition attributes, substitutions, closure evidence, sequent snapshots or opaque sequent tokens, and replay digests. This design allows certificates to be stored, inspected, compressed, corrupted for testing, and independently checked.

Third, the framework aims to support conservative replay. The checker does not attempt to reconstruct information that is not present in the certificate. When sufficient textual proof-state information is available, the checker performs schema-level replay for supported rule families. When only partial proof-state information is available, the checker falls back to conservative trace-shape validation and reports this replay mode explicitly. Unsupported rules, malformed side conditions, invalid substitutions, inconsistent branch closure evidence, or digest mismatches are rejected.

Fourth, the framework aims to preserve checkability under compression. Large proof traces can be compressed into macro certificates, but compression is not trusted as a storage-only optimization. The checker validates compressed macro steps against replay metadata and digest evidence. Therefore, a compressed certificate is accepted only when its macro replay is consistent with the corresponding full replay structure.

3.2 | Workflow Overview

The proposed workflow consists of four stages: proof production, certificate extraction, certificate compression, and independent checking. In the proof production stage, the input to the framework is a Rust program together with its specification. The source-level verification engine is invoked to construct a proof of the corresponding verification condition. This stage may rely on symbolic execution, proof search heuristics, large rule libraries, arithmetic simplification, and verifier-specific automation. The framework does not require this stage to be trusted. Its role is only to produce candidate proof evidence. In the certificate extraction stage, the generated proof trace is translated into an explicit JSON certificate. Each certificate step records the applied rule, its position in the proof tree, branch information, side-condition metadata, substitution information, closure evidence, and replay digest. When textual sequents are available, they are included in the certificate. When textual sequents are not available, stable opaque sequent tokens are used and the certificate is checked under conservative trace-shape replay.

In the certificate compression stage, the full certificate can optionally be compressed by grouping replayable proof fragments into macro steps. Compression reduces certificate size while preserving the metadata required for independent checking. The compressed certificate records macro-step boundaries, replay digests, and the information needed to validate that the macro step has not introduced unsupported or inconsistent proof evidence.

In the independent checking stage, the checker reads either a full or compressed certificate and validates it without calling the original proof engine. The checker verifies certificate structure,

known rule schemas, parent-child dependencies, branch metadata, side conditions, substitutions, closure evidence, and replay digests. The output is either accepted with replay statistics or rejected with an explicit diagnostic reason.

3.3 | Trusted and Untrusted Components

A key feature of the framework is the explicit separation between trusted and untrusted components. The trusted computing base consists of the small certificate checker, the JSON certificate reader, the rule-schema whitelist, conservative side-condition and substitution validators, branch-closure validation, and digest validation. These components are responsible for the final accept/reject decision. In the current implementation, the checker is intentionally compact: the reported trusted checker size is 205 lines of code across the benchmarked examples.

The untrusted side includes the original verification engine, proof-search heuristics, large rule-execution machinery, arithmetic simplification procedures inside the proof producer, source-language front-end processing, certificate generation scripts, and the certificate compressor. These components may help produce proof evidence, but their correctness is not assumed by the final checker.

The checker is conservative. If a certificate contains an unknown rule, malformed branch structure, invalid closure evidence, corrupted substitution, missing side condition, inconsistent initial sequent, digest mismatch, or invalid compressed macro step, the checker rejects the certificate. This behavior is confirmed by negative tests in which corrupted rule identifiers, substitutions, side conditions, initial sequents, branch closure evidence, and compressed macro steps are all rejected with explicit diagnostic categories. This separation gives the framework a proof-carrying structure: the verifier may be large, heuristic, and implementation-heavy, but the final trust decision is made by a small checker over explicit proof evidence.

3.4 | Running Example

We illustrate the workflow using a simple source-level verification example involving assignment and primitive-state updates. The input program contains a small Rust fragment together with a specification over its final state. The verification engine first proves the verification condition and emits a proof trace. The framework then converts this trace into a full certificate.

A simplified certificate step has the following structure:

```
{
  "id": "step-0",
  "rule": "assignment_update",
  "parent": null,
  "branch": "main",
  "side_conditions": {},
  "substitutions": {},
  "before": "...",
  "after": "...",
  "digest": "..."
}
```

The checker validates that the rule identifier belongs to a supported rule family, that the step is connected to the expected proof state, that the branch metadata is well formed, and that the replay digest is consistent with the certificate content. Subsequent steps are replayed in the same manner until the certificate reaches a closed proof state. If all branches are closed and all certificate-local checks succeed, the checker accepts the certificate.

For the primitive assignment example, the generated proof contains 20 rule steps, one branch, and one final closed goal. The full certificate size is 16.478 KB, while the compressed certificate size is 7.925 KB, giving a compression ratio of 2.039. The full certificate is checked in 0.625 ms and the compressed certificate is checked in 0.181 ms. This example demonstrates the basic case where proof evidence can be extracted, compressed, and independently replayed by the small checker.

The same workflow also scales to larger examples. For a binary-search example, the proof contains 4254 rule steps, 37 branches, and 37 final closed goals. The full certificate size is 2974.072 KB and the compressed certificate size is 488.672 KB, corresponding to a compression ratio of 5.713. The checker validates the full certificate in 83.348 ms and the compressed certificate in 1.422 ms. These results suggest that the framework can preserve independent checkability while substantially reducing the size and checking cost of large proof traces. Finally, the robustness of the checking boundary is illustrated by corrupted certificates. For example, replacing a valid rule identifier with an unsupported one leads to an UnsupportedRule rejection; corrupting a substitution causes a ReplayDigestMismatch; removing a required side condition causes a MalformedSideCondition; changing the initial sequent causes an InitialSequentMismatch; corrupting branch closure evidence causes an InvalidBranchClosingCondition; and corrupting a compressed macro step causes a macro-replay rejection. Therefore, the checker does not merely parse the certificate format; it enforces certificate integrity and rejects malformed proof evidence.

In addition to reporting replay statistics, this running example clarifies the role of each certificate component in the overall trust argument. The proof producer is responsible only for generating candidate proof evidence, whereas the checker is responsible for validating whether that evidence is structurally and locally replayable. The `rule` field determines which replay schema is used; the `parent` and `branch` fields determine the proof-tree topology; the `side_conditions` and `substitutions` fields expose rule-specific assumptions that would otherwise remain implicit inside the proof engine; and the `digest` field binds the replay-relevant content of the step to the surrounding proof trace.

4 | Certificate Design

This section describes the certificate representation used by our proof-carrying source-level verification workflow. The certificate is designed to serve as an explicit, inspectable, and independently replayable proof object. It is not merely an execution log emitted by the proof producer. Instead, it records the information required by a small checker to reconstruct the proof structure, validate rule applications at the supported replay level, check branch closure, and reject malformed or corrupted evidence. The design follows three principles. First, the certificate must separate proof production from proof trust: the original verification engine may generate the certificate, but the final accept/reject decision is made by the independent checker. Second, the certificate must

expose enough metadata to make failures diagnosable. A rejected certificate should indicate whether the failure is caused by an unsupported rule, malformed side condition, invalid substitution, broken branch closure, or digest mismatch.

4.1 | Full Proof Certificate Format

A full proof certificate is represented as a JSON object that records the essential evidence required for independent replay. It contains proof-level metadata, initial proof-state information, final proof-state summaries, and an ordered sequence of proof steps. The certificate is intended to be inspected and checked outside the original verification engine. Therefore, its top-level structure explicitly separates producer information, proof metadata, step-level replay evidence, branch metadata, and global replay summaries. The schematic structure is shown below.

```
{
  "certificate_version": "1.0",
  "producer": {
    "name": "rustydl-cert",
    "mode": "from-proof",
    "source": "example5.proof"
  },
  "proof": {
    "example": "example5",
    "rust_feature": "primitive/assignment",
    "initial_sequent": "...",
    "final_status": "closed"
  },
  "steps": [
    {
      "id": "s0",
      "rule": "assignment_update",
      "parent": null,
      "branch": "b0",
      "before": "...",
      "after": "...",
      "side_conditions": {},
      "substitutions": {},
      "closure": null,
      "digest": "..."
    }
  ],
  "branches": [
    {
      "id": "b0",
      "root": "s0",
      "closed": true,
      "closing_step": "s19"
    }
  ],
  "replay": {
    "mode": "precise",
    "root_digest": "...",
    "num_steps": 20,
    "num_branches": 1
  }
}
```

The field `certificate_version` fixes the certificate schema expected by the checker. The `producer` block records how the certificate was generated, but this block is not trusted for correctness. The `proof` block records the verification example, the source-level feature being checked, and the initial proof state. The `steps` array is the main replay object: each element represents one proof-rule application or one proof-trace event. The `branches` array records the proof-tree structure and final closure status. Finally, the `replay` block summarizes the replay mode and global digest information.

The checker treats the certificate as untrusted input. It first validates the schema, then checks that all referenced step identifiers, branch identifiers, parents, and closing steps are well formed. It then replays the proof according to the supported replay mode. If the certificate contains an unknown rule, malformed proof-tree structure, inconsistent branch metadata, or invalid digest, the checker rejects it.

4.2 | Proof-Step Representation

Each proof step records a local transition in the proof object. A step has a stable identifier, a rule name, a parent relation, a branch identifier, optional before/after sequent snapshots, side-condition evidence, substitution evidence, closure metadata, and a replay digest. A typical step has the following form.

```
{
  "id": "s42",
  "rule": "variable_substitution",
  "parent": "s41",
  "branch": "b3",
  "before": "Gamma ==> Delta",
  "after": "Gamma' ==> Delta'",
  "side_conditions": {
    "fresh": ["x_1"],
    "well_formed": true
  },
  "substitutions": {
    "x": "x_1"
  },
  "closure": null,
  "digest": "sha256:..."
}
```

The `id` field provides a stable reference for parent-child linking and diagnostics. The `rule` field is matched against the checker's rule-schema whitelist. The checker does not silently accept unknown rule names; unsupported rules produce an explicit rejection. The `parent` field specifies the previous step in the current proof branch, while the `branch` field identifies the branch to which the step belongs.

The `before` and `after` fields record sequent snapshots when they are available from the proof trace. In some traces, complete textual sequents are not exposed by the proof producer. In that case, the certificate records stable opaque sequent tokens and the checker reports conservative trace-shape replay rather than claiming full semantic replay. This distinction is important because the checker must not reconstruct information that the proof trace did not actually provide.

The `side_conditions` and `substitutions` fields store rule-specific evidence. For example, a substitution rule may require a concrete mapping from program variables to terms, while a branch-closing rule may require a reference to a matching antecedent or closure marker. The checker validates these fields

conservatively. Missing or malformed evidence causes rejection rather than fallback acceptance.

4.3 | Branch and Closure Metadata

Deductive verification proofs often contain branching structure. A certificate therefore cannot be checked as a flat sequence of rule names alone. It must also record how proof branches are created, how steps are assigned to branches, and why each final branch is considered closed.

Each branch descriptor contains a branch identifier, its root step, its parent branch if applicable, and its closure status.

```
{
  "id": "b7",
  "parent_branch": "b2",
  "root": "s103",
  "closed": true,
  "closing_step": "s148",
  "closure_reason": {
    "kind": "assumption",
    "antecedent": "x == x",
    "succedent": "x == x"
  }
}
```

The checker enforces three branch invariants. First, every proof step must belong to an existing branch. Second, parent-child step links must remain within the same branch unless the rule explicitly creates child branches. Third, every branch marked as closed must contain valid closure evidence. A branch cannot be declared closed solely because the proof producer says so.

Closure metadata may take several forms. For syntactic closure, the certificate records a pair of matching antecedent and succedent formulas, or an explicit closure marker emitted by the proof trace. For assumption-based closure, the certificate records the referenced assumption or sequent component. For unsupported closure styles, the checker rejects the certificate. This design is intentionally conservative. A certificate with a wrong closing step, missing closure witness, or inconsistent branch state is rejected even if the rest of the proof trace appears well formed.

4.4 | Side Conditions and Substitutions

Many proof rules are sound only under side conditions. For example, a variable substitution rule may require freshness, an update rule may require a well-formed assignment target, a borrowing rule may require a valid reference state, and an arithmetic simplification rule may require membership in a supported deterministic simplification family. The certificate therefore records side-condition evidence explicitly instead of leaving it implicit inside the proof producer.

Side conditions are represented as structured key-value objects.

```
{
  "side_conditions": {
    "fresh_variables": ["tmp_0"],
    "target_is_assignable": true,
    "rule_family": "assignment",
    "deterministic": true
  }
}
```

The checker validates side conditions according to the supported rule schema. For some rules, this means checking the presence and shape of required fields. For deterministic simplification rules, this means checking that the rule belongs to a supported simplification family. For rules whose side conditions cannot yet be independently reconstructed from the certificate, the checker rejects or classifies the step under a conservative replay mode, depending on the available evidence.

Substitutions are also represented explicitly.

```
{
  "substitutions": {
    "i": "i + 1",
    "arr": "arr'",
    "result": "mid"
  }
}
```

The checker verifies that substitution entries are well formed and are bound to the corresponding proof step. It also includes substitutions in the replay digest, so that a corrupted substitution changes the expected digest. This is important because substitution errors are a common way for an invalid proof trace to appear superficially plausible. In our negative tests, corrupted substitutions are rejected by digest mismatch, while missing side-condition evidence is rejected as malformed side-condition metadata.

4.5 | Replay Digests

Replay digests provide certificate-local integrity checks. Each step digest is computed from the replay-relevant content of the step, including the rule name, parent identifier, branch identifier, before/after state or opaque state token, side conditions, substitutions, and closure metadata. A simplified digest input can be viewed as follows.

```
digest_i =
  H(rule_i,
    parent_i,
    branch_i,
    before_i,
    after_i,
    side_conditions_i,
    substitutions_i,
    closure_i,
    digest_parent)
```

The inclusion of the parent digest makes the proof trace tamper-evident. A local change to an early step propagates to later digest checks. The final `root_digest` stored in the replay block summarizes the accepted proof object. During checking, the digest is recomputed from the certificate content rather than trusted as a producer-supplied value.

Digests do not replace rule checking. They only ensure that the replayed certificate is the same object that the checker is validating. Rule soundness is still enforced by the rule-schema replay discipline, side-condition validation, substitution validation, and branch-closure checks. Thus, a certificate with consistent digests but unsupported rules is rejected, and a certificate with a supported rule but corrupted replay-relevant content is also rejected.

Replay digests are especially useful for regression tests and artifact evaluation. They make small corruptions observable and

provide stable failure modes such as `ReplayDigestMismatch`, rather than allowing corrupted proof objects to fail only indirectly at a later stage.

4.6 | Compressed Certificate Format

Full certificates are useful for auditability, but large source-level proofs may contain thousands of proof steps. To reduce storage and checking overhead, the framework also supports compressed certificates. A compressed certificate replaces selected sequences of proof steps with macro steps while preserving enough metadata for independent replay.

A compressed macro step has the following schematic form.

```
{
  "id": "m12",
  "kind": "macro",
  "macro_rule": "deterministic_simplification_block",
  "branch": "b4",
  "first_step": "s201",
  "last_step": "s248",
  "num_elided_steps": 48,
  "input_digest": "sha256:...",
  "output_digest": "sha256:...",
  "macro_digest": "sha256:...",
  "summary": {
    "allowed_rule_families": [
      "polySimp",
      "inEqSimp",
      "literal_simplification"
    ],
    "deterministic": true
  }
}
```

The key point is that compression is not trusted. The checker does not accept a macro step merely because it claims to summarize a proof fragment. Instead, the macro must state its replay boundary, allowed rule families, input and output digests, and deterministic replay constraints. If the macro contains a non-deterministic or unsupported rule family, the compressed certificate is rejected.

The compressed format is therefore a proof object in its own right. It preserves the same top-level metadata as the full certificate, but the `steps` array may contain both ordinary steps and macro steps. The checker validates ordinary steps as before and validates macro steps using the macro replay discipline. A compressed certificate is accepted only when all ordinary and macro steps satisfy the checker's replay rules and the final digest is consistent. This design makes certificate compression a performance optimization rather than a trusted proof transformation. The full certificate remains the most explicit artifact, while the compressed certificate provides a smaller replayable object. In the evaluated examples, compression substantially reduces certificate size and checking time, while corrupted macro steps are rejected by the checker.

5 | Small-Kernel Certificate Checking

This section presents the small-kernel certificate checker used in our proof-carrying verification workflow. The main purpose of the checker is to separate proof production from proof trust.

TABLE 1 | Main components of the small-kernel certificate checker.

Component	Responsibility
Certificate parser	Reads the JSON certificate and checks that all required fields are present.
Structure validator	Checks step identifiers, parent-child dependencies, branch identifiers, and final goal status.
Rule-schema validator	Checks whether each proof step belongs to a supported rule family.
Side-condition checker	Validates side conditions required by supported proof rules.
Substitution checker	Checks certificate-local substitution records and their consistency with replay metadata.
Digest validator	Detects modifications of rule names, sequents, substitutions, side conditions, or macro-step contents.
Closure validator	Checks whether every reported final branch is closed by valid closure evidence.

The original source-level verifier is treated as an untrusted proof producer: it may search for proofs, apply automation, simplify formulas, and generate proof traces. The final verification decision, however, is made by an independent checker over an explicit certificate.

The checker is intentionally small and conservative. It does not perform proof search, does not invoke the original verifier, and does not reconstruct the full internal proof state of the proof producer. Instead, it validates the exported certificate by checking rule identifiers, proof-step dependencies, branch structure, side conditions, substitutions, replay digests, and closure evidence. If any required component is missing, unsupported, or inconsistent, the certificate is rejected.

5.1 | Checker Architecture

The checker accepts two kinds of certificates: full certificates and compressed certificates. A full certificate records the proof at the level of individual rule applications. A compressed certificate replaces selected proof fragments with macro steps while preserving enough replay information for independent checking.

The checking process consists of four main stages. First, the checker parses the JSON certificate and validates its basic format. Second, it checks the global proof structure, including step identifiers, parent-child dependencies, branch identifiers, and final goal status. Third, it replays each step according to the supported replay mode. Finally, it validates branch closure and returns either an acceptance result or a rejection diagnostic.

Table 1 summarizes the main components of the checker.

The checker maintains only lightweight replay state. This state records the current branch, the previously checked proof step, the rule family, local side-condition information, substitution metadata, and replay digests. The checker does not trust the original proof engine's internal data structures. This design keeps the trusted component small and makes the replay procedure auditable.

5.2 | Precise Replay

Precise replay is used when the certificate contains sufficient information to validate each proof step directly against the supported rule schema. In this mode, each proof step must provide

TABLE 2 | Supported rule families in schema-level textual replay.

Rule family	Checked evidence
Assignment/update	Rule identifier, substitution record, before-after replay digest.
Borrowing/reference	Rule identifier, reference-related side conditions, replay digest.
Mutable write	Rule identifier, write metadata, substitution record, replay digest.
Array/tuple/enum	Rule identifier, constructor or projection metadata when available.
Loop invariant	Rule identifier, branch information, invariant-related side conditions.
Arithmetic simplification	Recognized arithmetic rule family, local simplification metadata, digest.
Branch closure	Branch identifier, closing condition, closure marker, digest.

a rule identifier, a parent step, branch metadata, side-condition records when required, substitution records when required, and a replay digest.

For a proof step s_i, the checker validates the following conditions:

$$\begin{aligned}\mathrm{ValidStep}(s_i) = \mathrm{KnownRule}(s_i) \wedge \mathrm{ValidParent}(s_i) \\ \wedge \mathrm{ValidBranch}(s_i) \wedge \mathrm{ValidLocalEvidence}(s_i).\end{aligned} \tag{1}$$

Here, KnownRule checks that the rule belongs to the supported rule schema, ValidParent checks that the parent step has already been validated, ValidBranch checks that the step occurs in a valid branch context, and ValidLocalEvidence checks side conditions, substitutions, closure metadata, and digest consistency.

A certificate is accepted only if all proof steps are valid and all final goals are closed:

$$\mathrm{Accept}(C) \iff \Big(\forall s_i \in C.\, \mathrm{ValidStep}(s_i)\Big) \wedge \mathrm{AllBranchesClosed}(C). \tag{2}$$

This replay mode provides the strongest checking result in the current implementation. It is used for proof fragments where the certificate exposes enough local evidence to validate the replay without relying on the original proof producer.

5.3 | Schema-Level Textual Replay

When textual sequents are available, the checker performs schema-level textual replay. In this mode, the checker does not attempt to reproduce every internal operation of the original verifier. Instead, it checks whether the textual proof step is compatible with a supported rule family and with the local evidence recorded in the certificate.

The supported rule families include assignment and update rules, borrowing rules, mutable-reference rules, reference-write rules, array rules, tuple rules, enum rules, loop-invariant rules, deterministic simplification rules, arithmetic normalization rules, and branch-closing rules. These rule families are not trusted merely by name. The checker also validates the required local evidence, such as side conditions, substitutions, branch identifiers, and digests.

Table 2 gives a compact summary of the rule families checked by the current implementation.

This mode is useful because many existing verification engines can emit human-readable proof traces or textual proof-state snapshots, but these traces are often not designed as fully semantic proof objects. Schema-level replay therefore provides a practical middle ground: it checks more than a syntactic log while avoiding a full reimplementation of the original verifier.

5.4 | Conservative Trace-Shape Replay

Some proof traces do not expose full textual sequents for every intermediate proof state. In such cases, the checker cannot reconstruct the complete logical transformation performed by the proof producer. The checker therefore uses a separate conservative trace-shape replay mode.

In trace-shape replay, the checker validates the information that is available: known rule families, parent-child dependencies, branch structure, step identifiers, closure evidence, and replay digests. Proof states may be represented by stable opaque sequent tokens rather than full formulas. These tokens are not interpreted as logical formulas; they are used only to ensure that the exported trace has not been modified.

This mode is conservative. It does not claim to provide full semantic replay of hidden proof states. At the same time, it is not an accept-all mode. Unknown rules, malformed branch structures, inconsistent closure evidence, and digest mismatches still cause rejection. The replay result explicitly records that the certificate was checked under trace-shape validation rather than precise textual replay.

5.5 | Conservative Arithmetic Replay

Arithmetic reasoning is also handled conservatively. The checker recognizes a restricted set of arithmetic simplification rule families, such as polynomial simplification, polynomial division, inequality simplification, and literal normalization rules. These steps are checked through their rule names, local metadata, and replay digests.

The checker does not include a full SMT solver or a complete arithmetic decision procedure. This is a deliberate design choice. Adding a powerful arithmetic backend would increase automation, but it would also enlarge the trusted computing base. Instead, the checker accepts only arithmetic steps that are represented by supported certificate-level evidence.

For an arithmetic step s_i, the checker requires:

$$\begin{aligned}\mathrm{ValidArith}(s_i) = \mathrm{SupportedArithRule}(s_i) \\ \wedge \mathrm{ValidArithMetadata}(s_i) \wedge \mathrm{ValidDigest}(s_i).\end{aligned} \tag{3}$$

Arithmetic rules outside the supported replay discipline are rejected. Future versions of the framework could add independently checkable SMT-style arithmetic certificates, but such a backend should itself remain small and auditable.

5.6 | Failure Diagnostics

The checker is designed to fail closed. Invalid certificates are rejected with explicit diagnostic categories rather than silent failure or generic error messages. These diagnostics help users identify

TABLE 3 | Representative certificate rejection diagnostics.

Diagnostic	Meaning
`UnsupportedRule`	The certificate uses a rule identifier that is not in the supported rule schema.
`ReplayDigestMismatch`	The certificate-local replay data has been modified or is inconsistent.
`MalformedSideCondition`	A required side condition is missing or malformed.
`InitialSequentMismatch`	The certificate does not match the supplied initial sequent.
`InvalidBranchClosingCondition`	A proof branch is closed using invalid or inconsistent closure evidence.
`NonDeterministicRuleInSimplificationMacro`	A compressed macro step contains a rule that is not valid for that macro replay mode.

TABLE 4 | Trusted computing base boundary.

Component	Trust status
Certificate checker	Trusted. It makes the final accept-or-reject decision.
JSON certificate reader	Trusted as part of certificate parsing.
Rule-schema whitelist	Trusted. It defines the rule families accepted by the checker.
Digest validation	Trusted. It detects certificate modifications.
Side-condition and substitution checks	Trusted. They validate certificate-local proof evidence.
Original proof engine	Untrusted. It only produces candidate proof evidence.
Proof-search heuristics	Untrusted. They may find proofs but do not decide final validity.
Large rule engine or taclet machinery	Untrusted during checking. It is not invoked by the checker.
Certificate compressor	Untrusted. Compressed certificates must still be checked independently.
Arithmetic simplification engine	Untrusted unless its output is represented by checkable certificate evidence.

whether the failure is caused by an unsupported rule, malformed evidence, corrupted replay data, or invalid branch closure.

Table 3 lists the main diagnostic categories used by the current checker. These diagnostics are important for evaluating the checker. In the negative tests, corrupted rule identifiers, substitutions, side conditions, initial sequents, branch closure records, and compressed macro steps are all rejected. This shows that the checker is not merely validating the outer certificate format. It enforces the replay discipline over proof-local evidence.

5.7 | Trusted Computing Base

The trusted computing base is the part of the system that must be trusted for the final verification result. In our framework, the trusted computing base is limited to the certificate checker and the small amount of certificate infrastructure needed by the checker.

Table 4 summarizes the trusted and untrusted parts of the workflow.

This boundary is the main advantage of the proposed design. A conventional verifier-centered workflow requires users to trust the whole verification engine. Our workflow reduces the final trusted component to a small checker and its explicit certificate interface. The original verifier is still useful, but it is used only as a proof producer.

The current implementation demonstrates this design on certificates generated from several source-level verification examples, including assignment, borrowing, mutable references, tuples, arrays, enums, arithmetic, loops, and binary-search-style proofs. The same checker also validates compressed certificates and rejects corrupted certificates in negative tests. Therefore, the checker provides a practical trust-reduction layer for source-level verification without replacing the original proof producer.

6 | Certificate Compression

A full proof certificate records proof evidence at the granularity of individual replay steps. This representation is useful for auditing, debugging, and negative testing, but it can become large for proofs that contain thousands of deterministic symbolic-simplification steps. In particular, source-level verification proofs often contain long stretches of local rewriting, normalization, arithmetic simplification, and proof-state bookkeeping. These steps are individually simple, but recording each of them as a separate certificate node increases certificate size and replay overhead.

To address this issue, our framework supports compressed certificates. The goal of compression is not merely to reduce storage cost, but to preserve independent checkability. A compressed certificate must therefore remain replayable by the small checker without invoking the original verification engine. The checker does not trust the compression procedure itself. Instead, compression produces macro steps that summarize deterministic proof fragments, and the checker validates that each macro step satisfies the replay discipline before accepting the compressed certificate.

6.1 | Motivation

The motivation for certificate compression comes from a practical tension in proof-carrying verification. On the one hand, certificates should be explicit enough to make the final verification result independently checkable. On the other hand, fully explicit certificates may contain large amounts of repetitive low-level evidence. This is especially common in source-level verification, where a single user-visible proof rule may trigger many internal simplification steps before a branch is closed.

A monolithic verifier can hide these internal steps inside its proof engine. However, doing so would defeat the purpose of proof-carrying verification, because the small checker would then have to trust the large verifier's internal execution. Our design instead keeps the proof evidence external, but allows deterministic fragments to be represented as checked macro steps. Thus, compression reduces the size of the certificate while preserving the principle that the final accept/reject decision belongs to the independent checker.

Let a full certificate be represented as a sequence of replay steps

$$C = \langle s_0, s_1, \dots, s_n \rangle,$$

where each step records a rule identifier, parent relation, branch identifier, side-condition data, substitution information, and replay digest. A compressed certificate replaces selected contiguous fragments

$$\langle s_i, s_{i+1}, \dots, s_j \rangle$$

with a macro step

$$m_{i:j}.$$

The macro step is accepted only if it summarizes a deterministic replay fragment whose boundary states, rule families, branch metadata, and digest evidence are consistent with the original replay discipline.

6.2 | Macro-Step Construction

Macro steps are constructed from proof fragments that satisfy three conditions. First, the fragment must be local to a single replay context, meaning that it does not merge unrelated branches or invent new branch closures. Second, the fragment must contain only supported deterministic rule families, such as symbolic simplification, arithmetic normalization, syntactic rewriting, and other checker-recognized replay steps. Third, the fragment must expose enough boundary evidence for the checker to validate the transition from the input state of the macro step to its output state.
A compressed macro step has the following abstract form:

$$m = (id, kind, parent, branch, rules, in, out, digest).$$

Here, *id* is the macro-step identifier, *kind* records the macro category, *parent* records the dependency on the preceding proof state, *branch* records the branch context, *rules* summarizes the rule families contained in the macro, *in* and *out* are the boundary replay states, and *digest* is the integrity evidence used during compressed replay.
The construction deliberately excludes proof fragments that contain non-deterministic proof search, unsupported rule families, or branch-closing evidence that must be checked explicitly. Such steps remain explicit in the compressed certificate. This design prevents compression from hiding proof obligations that are semantically important for sound replay. In particular, branch closure, side-condition validation, and unsupported rules cannot be silently absorbed into a macro step.
The resulting compressed certificate can be viewed as a mixed representation:

$$C^\star = \langle s_0, m_{1:k}, s_{k+1}, m_{k+2:\ell}, \dots, s_t \rangle,$$

where ordinary steps and macro steps coexist. Ordinary steps are checked using the full replay procedure, while macro steps are checked using the compression-preserving replay procedure described below.

6.3 | Compression-Preserving Replay

Compression-preserving replay ensures that checking a compressed certificate does not require trusting the compressor. The small checker treats every macro step as a proof obligation. For each macro step, the checker validates its boundary state, branch context, rule-family summary, and digest. If any of these components is inconsistent with the replay state, the compressed certificate is rejected.
The replay judgment for ordinary certificate steps has the form

$$\Gamma \vdash s_i \rightsquigarrow s_{i+1},$$

where Γ denotes the supported rule-schema environment. For a macro step, the checker uses a compressed replay judgment

$$\Gamma \vdash s_i m_{i:j} s_j.$$

This judgment does not mean that the checker blindly trusts the macro. Rather, it means that the macro contains sufficient evidence for the checker to validate that the summarized fragment is admissible under the supported replay discipline.
The checker enforces the following conditions for compressed replay:

(1) the macro input matches the current replay state,
(2) the macro output matches the advertised successor state,
(3) all summarized rule families are supported,
(4) no hidden nondeterministic or closing rules,
(5) branch identity is preserved unless explicitly updated,
(6) the replay digest is consistent with the macro payload.

If these conditions hold, the macro step is accepted as a checked abbreviation of a replayable proof fragment. Otherwise, the checker rejects the certificate.
This design gives the following compression-preservation property:

$$\mathsf{CheckCompressed}(C^\star) = \mathsf{accept} \Rightarrow \mathsf{Replayable}(C^\star, \Gamma).$$

That is, acceptance of a compressed certificate implies that every ordinary step and every macro step is replayable under the checker-supported rule schemas. The guarantee is intentionally conservative. The checker may reject some compressible proof fragments if they contain insufficient replay evidence, but it should not accept a macro step whose replay evidence is malformed or semantically unsupported.
In the implementation, this design substantially reduces certificate size for large proofs while preserving independent checking. For example, the binary-search certificate is reduced from several megabytes to a much smaller compressed certificate, while the compressed certificate remains checkable by the same small trusted checker. The compressed checker also avoids replaying every low-level simplification step individually, which reduces replay time for large proof traces.

6.4 | Rejection of Invalid Macro Certificates

A key requirement of compressed certificates is that compression must not become a new trusted component. Therefore, the checker must reject invalid macro certificates rather than accepting them as opaque proof summaries. We validate this behavior using negative tests that deliberately corrupt compressed macro steps.

Invalid macro certificates may arise in several ways. A macro step may claim to summarize a deterministic simplification fragment while actually containing a non-deterministic rule. It may advertise an incorrect boundary state, use a wrong branch identifier, contain a corrupted digest, or hide a rule that should have been checked explicitly. Any of these cases breaks the contract between compression and replay.

The checker rejects such certificates using explicit diagnostic categories. For example, if a macro step contains a rule that is not admissible inside a simplification macro, the checker reports a macro-level replay error instead of accepting the compressed proof. This behavior is important because otherwise a malicious or buggy compressor could hide invalid proof steps inside a macro and bypass the small checker.

Formally, let m be a macro step in a compressed certificate. The checker requires

$$\mathsf{MacroValid}(m, \Gamma, s_{in}, s_{out}).$$

If this predicate fails, then compressed replay fails:

$$\neg\mathsf{MacroValid}(m, \Gamma, s_{in}, s_{out}) \Rightarrow \mathsf{CheckCompressed}(C^{\star}) = \mathsf{reject}.$$

Thus, macro steps are not trusted proof objects. They are compact proof obligations whose validity must be established by the checker.

This rejection behavior is central to the trusted-computing-base reduction of the framework. The compressor may be useful for performance and storage, but it is not part of the trusted core. The trusted decision remains with the small checker, which accepts compressed certificates only when their macro steps preserve the same replay discipline as full certificates.

7 | Implementation

This section describes the implementation of our proof-carrying source-level verification artifact. The implementation is designed as a practical software extension rather than as a monolithic verifier replacement. Its main purpose is to separate proof production from proof trust: a large verification backend may produce proof evidence, while a small independent checker validates the emitted certificate. The repository therefore contains certificate extraction, certificate compression, independent replay checking, command-line tooling, benchmark generation, negative-test generation, and reproducibility scripts.

7.1 | Repository Organization

The repository is organized around the certificate-based verification pipeline. The main implementation consists of an independent checker, certificate-format documentation, source-level verification examples, extracted proof traces, generated benchmark results, unit tests, a command-line entry point, and reproducibility targets. The checker is kept in a dedicated component so that the trusted part of the artifact can be inspected separately from proof-producing infrastructure. Documentation files describe the certificate format, the replay discipline, and the artifact usage workflow. Example programs and proof traces are separated from generated results, which makes it possible to rerun the experiments without overwriting the source inputs.

This organization reflects the central design principle of the artifact. Proof production, certificate extraction, certificate checking, compression, testing, and result reporting are implemented as separate stages. The original verification backend is therefore not treated as the final trusted authority. Instead, it is used to generate candidate proof evidence, while the independent checker performs the final validation. The repository also includes generated benchmark tables and negative-test reports, allowing users to inspect both successful replay cases and intentionally corrupted certificates.

7.2 | Certificate Extraction

Certificate extraction converts existing proof evidence into explicit JSON certificates. The extractor treats the original verifier as an untrusted proof producer. It does not assume that a successful proof file is itself a trusted verification result. Instead, it parses the proof trace and emits a replayable certificate containing rule names, step identifiers, parent-child dependencies, branch identifiers, substitutions, side conditions, sequent snapshots when available, closure information, and replay digests.

The generated full certificate records the proof as a sequence of explicit proof steps. Each step is associated with a rule schema and local replay information. When textual sequent information is available in the proof trace, it is stored in the certificate and used during schema-level replay. When the trace does not expose full textual sequents, the extractor records stable opaque sequent tokens and marks the certificate for conservative trace-shape replay. This conservative mode avoids reconstructing proof information that was not actually present in the original trace.

The extraction process is intentionally non-invasive. Rather than requiring deep changes inside the verification backend, the extractor operates on readable proof traces. This design makes the framework easier to apply to existing verification infrastructure and keeps the trusted checker independent from proof search, symbolic execution, source-language front-end processing, and backend-specific proof automation.

7.3 | Checker Implementation

The checker is implemented as a small replay kernel. It validates certificate structure, known rule schemas, step dependencies, branch metadata, side conditions, substitutions, closure evidence, and replay digests. The checker does not invoke the original proof-producing engine during validation. This is the central trusted-computing-base reduction of the artifact: the large verifier is used to discover a candidate proof, but the final accept/reject decision is made by the independent checker.

The checker supports multiple replay modes. In precise replay, it validates step identifiers, parent-child links, branch information, substitutions, side-condition attributes, closure markers, current-goal chaining, and replay digests. In schema-level textual replay, it validates supported proof-rule families against certificate-local evidence. In conservative trace-shape replay, it validates known rule families, proof-tree shape, branch closure, and digest consistency when full textual sequents are not available. Arithmetic

rules are handled conservatively through supported simplification families rather than by trusting a full external arithmetic engine.

Unsupported or malformed certificates are rejected explicitly. The checker does not silently accept unknown rules, invalid substitutions, missing side conditions, inconsistent branch closure evidence, or corrupted macro steps. The negative-test suite confirms this behavior by checking that corrupted rule identifiers, substitutions, side conditions, initial sequents, branch closures, and compressed macro steps are all rejected with diagnostic error categories.

The checker is deliberately small. In the reported benchmark configuration, the trusted checker contains 205 lines of code across all evaluated examples. Positive benchmark results show successful replay for primitive assignments, borrowing, mutable references, tuples, arrays, enums, arithmetic examples, loop-style examples, and binary search. This compact implementation is intended to make the trusted component auditable while still supporting the rule families required by the included verification examples.

7.4 | Command-Line Interface

The artifact provides a command-line interface that exposes the main certificate operations. The interface is designed to support both individual debugging and batch artifact evaluation. It includes operations for certificate extraction, certificate checking, proof-producing verification, certificate compression, and compressed-certificate checking. This allows users to run the complete workflow from source-level proof evidence to independently checked certificates.

The interface supports generating a full certificate from an existing proof trace, checking a full certificate independently, invoking the verification backend as a proof producer when available, compressing a full certificate into a macro certificate, and checking the compressed certificate. Each checking operation returns either acceptance with replay statistics or rejection with an explicit diagnostic reason. This makes the tool useful not only for successful artifact reproduction, but also for debugging malformed certificates and unsupported proof steps.

The command-line interface also separates proof production from proof validation. A user may first invoke the backend to generate proof evidence and then run the checker as a separate step. This separation is important because the artifact does not claim that the proof-producing backend itself is small or trusted. Instead, the final validation result is produced by the independent replay checker over the emitted certificate.

7.5 | Benchmark and Test Generation

The benchmark infrastructure evaluates both positive and negative cases. Positive benchmarks measure whether valid certificates are accepted and report replay statistics, certificate size, compressed certificate size, compression ratio, proof-production time, checker time, compressed-checker time, and trusted checker size. The benchmark suite includes examples covering primitive assignment, borrowing, mutable references, tuples, arrays, enums, arithmetic, loop-invariant proofs, larger loop-style proofs, and binary search. In the current results, all positive examples are successfully verified by the checker, and the final number of open goals is zero for every benchmarked example.

The implementation also generates negative certificates by deliberately corrupting valid proof evidence. These corruptions include unsupported rule identifiers, modified substitutions, missing side conditions, inconsistent initial sequents, invalid branch closure evidence, and corrupted compressed macro steps. The expected result for each negative case is rejection. The reported negative-test results show that all corrupted cases are rejected by the checker with explicit diagnostic errors.

Benchmark and test results are emitted in both CSV and Markdown formats. The CSV output supports downstream analysis and plotting, while the Markdown output provides a human-readable artifact report. This dual format helps readers inspect the results directly while also making the evaluation easy to process automatically.

7.6 | Reproducibility Support

The repository includes reproducibility support at three levels. First, the artifact provides explicit command-line operations for each stage of the pipeline: extracting a certificate, checking a certificate, compressing a certificate, checking a compressed certificate, and regenerating benchmark results. Second, the repository provides convenience targets for running unit tests and benchmark generation. Third, the generated results are stored in stable CSV and Markdown formats, making it easy to compare future runs with the reported artifact results.

The benchmark output records both proof-production and checking costs. For example, the larger loop-style example contains thousands of rule steps and multiple branches, while the binary-search example also contains thousands of replayed steps and dozens of branches. In both cases, the full certificates are accepted, the compressed certificates are accepted, and compressed checking is substantially faster than full checking. These results support the claim that certificate compression can reduce checking overhead while preserving independent replay validation.

These reproducibility features are important for the intended venue. Since *Software: Practice and Experience* emphasizes implemented systems, engineering practice, and evaluable artifacts, the implementation is structured so that readers can inspect the trusted checker, regenerate certificates, run positive and negative tests, reproduce benchmark tables, and verify that corrupted certificates are rejected rather than accidentally accepted.

8 | Evaluation

This section evaluates whether the proposed proof-carrying source-level verification framework achieves its intended goals: producing replayable proof certificates, checking them independently with a small trusted kernel, reducing certificate size through compression, and rejecting malformed proof evidence. The evaluation is based on the benchmark subjects included in the artifact, which cover primitive assignments, borrowing, mutable references, tuples, arrays, enums, loop invariants, arithmetic simplification, and binary-search-style verification traces.

TABLE 5 | Benchmark subjects and proof-structure statistics.

Subject	Feature	Rule steps	Branches	Open goals	Closed goals
example5	primitive/assignment	20	1	0	1
example6	borrowing	25	1	0	1
example7-and-8	mutable-reference/tuple	21	1	0	1
example1	loop-invariant	95	1	0	1
example2	array	24	1	0	1
example3	enum	1	1	0	1
example9	arithmetic	9	1	0	1
example10	loop/binary-style	11170	40	0	40
binary-search	binary-search	4254	37	0	37

8.1 | Research Questions

We organize the evaluation around the following research questions.

RQ1: Certificate generation. Can existing source-level verification traces be converted into explicit proof certificates that preserve enough structure for independent replay?

RQ2: Certificate size. How large are the generated certificates, and how much storage reduction is obtained by certificate compression?

RQ3: Checking cost. What is the runtime overhead of independently checking full and compressed certificates?

RQ4: Rule coverage. Which proof-rule families and replay modes are exercised by the benchmark subjects?

RQ5: Robustness against invalid evidence. Does the checker reject certificates whose rule identifiers, substitutions, side conditions, branch closures, initial sequents, or compressed macro steps are corrupted?

RQ6: Trusted computing base. How does the size of the trusted checker compare with the original verifier-centered workflow?

8.2 | Experimental Setup

The experimental workflow consists of four stages. First, the proof producer generates or provides proof traces for the benchmark verification subjects. Second, the certificate extractor converts each proof trace into a full JSON certificate. Third, the compression pass constructs a compressed certificate by grouping replayable proof fragments into macro steps while preserving replay metadata and digest evidence. Finally, the independent checker validates both the full and compressed certificates.

For each benchmark subject, we record the number of proof-rule steps, the number of branches, the number of final open and closed goals, the full certificate size, the compressed certificate size, the compression ratio, the proof production time, the full-certificate checking time, the compressed-certificate checking time, and the trusted checker size in lines of code. Checking time is measured separately from proof production time because the central goal of this work is not to improve proof search, but to reduce the trusted component required for accepting a verification result.

All benchmark subjects are expected to be successfully verified by the proof producer and then independently accepted by the checker. Negative tests are constructed by deliberately corrupting certificates. For those tests, the expected behavior is rejection with a diagnostic error category.

8.3 | Benchmark Subjects

Table 5 summarizes the benchmark subjects. The subjects are intentionally small to medium-sized source-level verification examples, but they cover different proof patterns: straight-line assignment, borrowing, mutable references, tuples, arrays, enums, loop invariants, arithmetic simplification, and binary-search-style branching verification.

All nine subjects are successfully verified and independently accepted. Across the benchmark suite, the checker replays 15,619 proof-rule steps and validates 84 final closed goals, with no remaining open goals. The largest proof is `example10`, which contains 11,170 rule steps and 40 final proof branches. The second largest subject is `binary-search`, which contains 4,254 rule steps and 37 final proof branches.

8.4 | Certificate Size and Compression Ratio

Table 6 reports the full and compressed certificate sizes. Full certificate sizes range from 1.733 KB for the enum example to 7,884.023 KB for the largest loop/binary-style example. Compressed certificate sizes range from 1.250 KB to 1,289.689 KB. The observed compression ratio ranges from 1.443× to 5.958×, with an average ratio of approximately 4.066×.

TABLE 6 | Certificate size and compression ratio.

Subject	Full cert. KB	Compressed cert. KB	Ratio
example5	16.478	7.925	2.039
example6	22.022	13.458	1.632
example7-and-8	17.152	3.475	4.883
example1	73.085	11.943	5.958
example2	16.199	3.599	4.259
example3	1.733	1.250	1.443
example9	10.663	2.278	4.914
example10	7884.023	1289.689	5.750
binary-search	2974.072	488.672	5.713
Mean	1223.936	202.477	4.066

The results show that compression is most effective on larger and more repetitive proof traces. For example, `example10` is reduced from 7,884.023 KB to 1,289.689 KB, and `binary-search` is reduced from 2,974.072 KB to 488.672 KB. These results suggest that macro-level compression is particularly useful for loop-heavy or branch-heavy verification traces, where many local proof patterns are repeated.

8.5 | Checking Time

Table 7 reports proof production time, full-certificate checking time, and compressed-certificate checking time. The full checker validates small examples in less than 3 ms and validates the largest benchmark in 217.890 ms. Compressed checking is consistently faster, ranging from 0.019 ms to 2.360 ms.

TABLE 7 | Proof production and certificate checking time.

Subject	Proof (ms)	Full check (ms)	Comp. check (ms)
example5	8561.902	0.625	0.181
example6	6833.951	0.808	0.310
example7-and-8	7047.690	0.610	0.048
example1	21782.466	2.046	0.055
example2	5931.490	0.983	0.041
example3	5943.207	0.075	0.019
example9	7047.465	0.324	0.026
example10	70229.821	217.890	2.360
binary-search	35470.265	83.348	1.422
Mean	18760.917	34.079	0.496

The key observation is that independent checking is much cheaper than proof production. For example, the `binary-search` subject requires 35,470.265 ms for proof production, while full certificate checking takes 83.348 ms and compressed checking takes 1.422 ms. Similarly, `example10` requires 70,229.821 ms for proof production, while compressed checking takes only 2.360 ms. This supports the intended separation between expensive proof search and lightweight proof trust.

8.6 | Rule Coverage and Replay Modes

The benchmark suite exercises the rule families needed by the included source-level verification examples. Table 8 summarizes the main rule categories and the replay mode used by the checker.

TABLE 8 | Rule coverage and replay modes.

Rule category	Replay mode	Representative subjects
Assignment/update	Precise replay	example5
Borrowing/reference	Schema-level replay	example6
Mutable reference/write	Schema-level replay	example7-and-8
Tuple rules	Schema-level replay	example7-and-8
Array rules	Schema-level replay	example2
Enum rules	Schema-level replay	example3
Loop invariant rules	Trace-shape and schema replay	example1, example10
Arithmetic simplification	Conservative arithmetic replay	example9
Branch closure	Precise closure validation	example10, binary-search
Compressed macro steps	Digest-based macro replay	all compressed subjects

The checker does not attempt to reimplement the full proof engine. Instead, it validates a conservative replay discipline over supported rule families. When sufficient information is available, the checker performs precise replay over rule identifiers, parent-child dependencies, branch metadata, substitutions, side conditions, closure evidence, and replay digests. When proof traces expose less textual sequent information, the checker falls back to conservative trace-shape validation, in which it checks the known rule family, proof-tree shape, branch structure, closure evidence, and digest consistency.

Arithmetic rules are handled conservatively. The checker validates supported arithmetic simplification families, but it does not trust an external SMT solver or the original arithmetic simplifier as part of the small trusted kernel. This design is intentionally conservative: unsupported or underspecified rule evidence is rejected rather than silently accepted.

8.7 | Negative Certificate Tests

To evaluate whether the checker rejects invalid proof evidence, we construct negative tests by corrupting different certificate components. Table 9 reports the expected result, actual result, and diagnostic category for each negative test.

All negative tests are rejected. These results are important because they show that the checker is not merely validating the presence of a certificate file or accepting arbitrary proof traces. It checks rule identifiers, digest consistency, side-condition structure, initial-state consistency, branch-closing evidence, and compressed macro-step validity. Therefore, certificate compression is not trusted as an unchecked optimization: corrupted compressed evidence is also detected and rejected.

8.8 | TCB Size Comparison

The proposed design reduces the trusted computing base by separating proof production from proof checking. In a conventional verifier-centered workflow, the final verification result depends on the full proof engine, including proof search, rule application machinery, simplification procedures, branch management, and implementation-specific proof-state manipulation. In contrast, our workflow treats the original verifier as an untrusted proof producer. The final accept/reject decision is made by a small independent checker.

Table 10 compares the trusted components in the two workflows. The measured trusted checker size is 205 lines of code across all benchmark subjects. This number should not be interpreted as a complete replacement for the original verifier. Rather, it reflects the size of the component that must be trusted for the final certificate acceptance decision in the evaluated artifact. The original verifier remains necessary for proof production, but it is no longer part of the final trusted boundary.

Overall, the evaluation supports the following conclusions. First, existing source-level verification traces can be converted into explicit replayable certificates. Second, certificate compression substantially reduces storage size, especially for large repetitive proofs. Third, independent checking is much faster than proof production, and compressed checking is faster still. Fourth, the checker rejects corrupted certificates with explicit diagnostic categories. Finally, the trusted boundary is reduced from a large verifier-centered stack to a small certificate checker and its associated replay discipline.

TABLE 9 | Negative certificate tests.

Negative case	Expected	Actual	Diagnostic category
Corrupted rule identifier	reject	reject	UnsupportedRule
Corrupted substitution	reject	reject	ReplayDigestMismatch
Missing side condition	reject	reject	MalformedSideCondition
Wrong initial sequent	reject	reject	InitialSequentMismatch
Wrong branch closure	reject	reject	InvalidBranchClosingCondition
Corrupted compressed macro step	reject	reject	NonDeterministicRuleInSimplificationMacro

TABLE 10 | Trusted computing base comparison.

Component	Verifier-centered workflow	Certificate-checking workflow
Source-level proof producer	Trusted	Untrusted
Proof search heuristics	Trusted	Untrusted
Large rule execution engine	Trusted	Untrusted
Arithmetic simplification engine	Trusted	Untrusted
Certificate extractor	Not applicable	Untrusted
Certificate compressor	Not applicable	Untrusted
JSON certificate parser	Not applicable	Trusted
Rule-schema whitelist	Not applicable	Trusted
Replay kernel	Not applicable	Trusted
Digest validation	Not applicable	Trusted
Branch-closure validation	Trusted inside verifier	Trusted inside checker
Measured trusted checker size	Full verifier stack	205 LOC

9 | Discussion

This section discusses the practical meaning of the proposed proof-carrying source-level verification workflow. The goal of the framework is not to replace an existing deductive verifier, nor to improve proof search automation directly. Instead, it changes where the final trust decision is made. A large verifier may still be used to discover proofs, apply symbolic rules, and discharge verification conditions, but the final acceptance of a verification result is delegated to a small certificate checker. This design is particularly suitable for software engineering settings in which verification results must be audited, reproduced, stored, or independently validated after proof generation.

9.1 | What Trust Is Reduced?

The main trust reduction achieved by the proposed workflow is the separation between proof production and proof acceptance. In a conventional verifier-centered workflow, the user must trust the entire verification stack, including proof search, rule application, simplification procedures, proof-state management, front-end transformations, and implementation-specific automation heuristics. In contrast, our workflow treats the verifier as an untrusted proof producer. The verifier is allowed to generate candidate proof evidence, but this evidence is accepted only if it can be replayed by an independent checker.

The trusted computing base is therefore shifted from a large verification engine to a compact certificate checker, together with its certificate parser, rule schema interface, digest validation, branch-checking discipline, and conservative side-condition validation. The original verifier, proof search heuristics, certificate extraction scripts, and certificate compression procedure are not trusted for the final result. If any of these components emit malformed, unsupported, or inconsistent evidence, the checker rejects the certificate.

This reduction is not a claim that the checker proves all semantic properties of the original programming language from first principles. Rather, it is a more practical engineering claim: the final verification result no longer depends on the full internal correctness of the proof-producing engine. The checker validates whether the emitted certificate conforms to an explicit replay discipline. This makes the trusted boundary smaller, more inspectable, and easier to test.

Table 11 summarizes the trust boundary before and after introducing proof-carrying certificate checking.

The empirical results support this intended boundary reduction. Across the included positive examples, the checker accepts certificates for primitive assignment, borrowing, mutable references, tuples, loops, arrays, enums, arithmetic, and binary-search-style verification cases. The trusted checker size reported in the benchmark is 205 lines of code, while the original proof-producing workflow remains outside the trusted boundary. This does not eliminate all trust, but it concentrates trust in a much smaller and more auditable component.

9.2 | Practical Use in Verification Workflows

The proposed workflow can be integrated into practical verification pipelines as a post-verification validation layer. A developer or verification engineer first runs a source-level verifier to produce a proof trace. The trace is then converted into a structured proof certificate. The certificate may be stored, compressed, transmitted, or reviewed independently. Finally, the small checker validates the certificate and returns an accept or reject result together with replay statistics.

This workflow is useful in several software engineering scenarios. First, it supports reproducible verification. Instead of relying only on the state of a large verifier installation, a project can archive the certificate associated with a verification result. Later, the result can be rechecked without rerunning the full proof-producing

TABLE 11 | Trust boundary comparison between conventional verifier-centered verification and the proposed proof-carrying workflow.

Component	Conventional workflow	Proposed workflow
Proof search engine	Trusted as part of the verifier result	Untrusted proof producer
Rule application machinery	Trusted inside the verifier	Rechecked through certificate replay when supported
Proof-state management	Trusted inside the verifier	Validated through certificate structure, parent-child links, branches, and closure metadata
Arithmetic simplification	Trusted as implemented by the verifier	Checked conservatively through supported arithmetic rule families
Certificate compression	Usually absent or tool-specific	Untrusted; compressed certificates must be replay-checked
Final accept/reject decision	Made by the full verifier	Made by the small certificate checker
Failure reporting	Verifier-specific failure mode	Explicit certificate rejection reason

environment. Second, it supports continuous integration. A verification pipeline can run the large verifier when proofs are generated or updated, but subsequent validation can be performed by the lightweight checker. Third, it supports independent auditing. A reviewer does not need to trust every implementation detail of the original verifier; instead, the reviewer can inspect the certificate format, the supported replay rules, and the checker behavior.

Certificate compression further improves the practicality of this workflow. The benchmark results show that large certificates can be substantially compressed while preserving replay checkability. For example, the binary-search example has a full certificate size of 2974.072 KB and a compressed certificate size of 488.672 KB, giving a compression ratio of 5.713. The larger loop/binary-style example has a full certificate size of 7884.023 KB and a compressed certificate size of 1289.689 KB, giving a compression ratio of 5.750. These results suggest that certificate storage overhead can be reduced without making compression a trusted step.

The checker is also fast enough for use as a validation step. In the benchmark, small examples are checked in less than a few milliseconds, while larger examples such as binary search are checked in 83.348 ms for full certificates and 1.422 ms for compressed certificates. The loop/binary-style example is checked in 217.890 ms for the full certificate and 2.360 ms for the compressed certificate. Thus, the independent checker can be used as a practical rechecking mechanism rather than only as a theoretical artifact.

9.3 | Comparison with Conventional Verifier Centered Workflows

Conventional deductive verification workflows are centered around a verifier that both produces and validates proofs. This architecture is convenient because all automation is contained inside one system. However, it also makes the trusted computing base large. If the verifier reports success, the user must trust that proof search, rule application, simplification, internal proof-state transformations, and final closure checks were all implemented correctly.

The proposed workflow keeps the automation benefits of the conventional approach but changes the trust model. The large verifier is still valuable because it performs proof search and generates proof evidence. However, the verifier is no longer the final authority. The final result is accepted only when the emitted certificate satisfies the replay discipline of the small checker. This design is similar in spirit to proof-carrying code and certifying compilation: a complex producer may be used for automation, but a smaller checker validates the evidence that matters for trust. Table 12 compares the two workflows from a software engineering perspective.

A key practical difference is failure behavior. In a verifier-centered workflow, a failed proof, corrupted proof state, or unsupported replay scenario may appear as a tool-specific error. In the proposed workflow, certificate validation returns explicit rejection reasons. The negative-test results demonstrate this behavior: corrupted rule identifiers, substitutions, missing side conditions, wrong initial sequents, invalid branch closure evidence, and corrupted compressed macro steps are all rejected with diagnostic categories. This behavior is important because the checker is not merely confirming that a proof file exists; it is enforcing a certificate integrity discipline.

At the same time, the proposed workflow should not be interpreted as a complete replacement for conventional verifiers. The large verifier remains necessary for proof discovery, automation, and interaction with source-level program semantics. The contribution is therefore complementary: existing verifiers can continue to serve as powerful proof producers, while the certificate checker provides a smaller and more transparent final validation layer.

9.4 | Threats to Validity

Several threats to validity should be considered when interpreting the current results.

Internal validity. The implementation is intentionally conservative, but the checker itself is still trusted. Bugs in the checker, certificate parser, digest computation, or rule-schema validation may affect the final result. The negative tests reduce this risk by corrupting multiple certificate components and checking that the checker rejects them. However, negative testing cannot prove the absence of all implementation errors. A future implementation could further reduce this threat by formalizing the checker or implementing the replay kernel in a proof assistant.

Construct validity. The reported measurements evaluate certificate size, compression ratio, replay time, rule-step counts, branch information, and rejection behavior. These metrics capture important aspects of a proof-carrying workflow, but they do not measure all dimensions of verification usefulness. For example, they do not directly measure developer effort, proof engineering cost, or the usability of certificate diagnostics. Therefore, the current evaluation supports the claim that certificate checking is feasible and lightweight, but not the stronger claim that the

TABLE 12 | Comparison between verifier-centered and certificate-centered verification workflows.

Aspect	Verifier-centered workflow	Certificate-centered workflow
Primary artifact	Verifier result or internal proof file	Explicit replayable certificate
Automation	Performed by the verifier	Performed by the verifier as an untrusted producer
Rechecking	Requires verifier-specific machinery	Performed by an independent checker
Auditability	Limited by verifier complexity	Improved through explicit certificate structure
Storage and reuse	Often tool-dependent	Certificates can be archived and replayed
Compression	Not usually part of the trust story	Compressed certificates are checked explicitly
Handling corrupted evidence	Depends on verifier internals	Rejected with diagnostic categories

workflow automatically improves developer productivity in all settings.

External validity. The current artifact targets a representative subset of source-level Rust verification examples, including assignments, borrowing, mutable references, tuples, arrays, enums, loops, arithmetic, and binary-search-style proofs. Nevertheless, it does not cover the full Rust language or all proof rules of the underlying verifier. Programs using unsupported language features or proof rules may produce certificates that the current checker rejects. This is an intentional conservative design choice, but it limits generalization beyond the supported examples.

Replay completeness. Some proof traces may not expose full textual sequents or complete semantic information. In such cases, the checker falls back to conservative trace-shape validation using stable opaque tokens, proof-tree structure, branch metadata, and replay digests. This mode is useful for checking the integrity of available proof evidence, but it is weaker than full semantic replay. The paper therefore distinguishes between precise replay, schema-level textual replay, conservative trace-shape replay, and conservative arithmetic replay. The results should be understood relative to the replay mode reported for each certificate.

Arithmetic reasoning. The current checker handles arithmetic conservatively through supported simplification rule families. It does not include a full SMT-backed arithmetic certificate checker. As a result, arithmetic-heavy verification tasks may require richer certificate evidence or an independently checkable arithmetic backend. This limitation does not invalidate the proof-carrying architecture, but it marks an important direction for extending the trusted replay kernel.

Producer dependence. Although the verifier is not trusted for the final accept/reject decision, the workflow still depends on the proof producer to generate useful proof evidence. If the producer emits incomplete traces, unsupported rules, or insufficient side-condition information, the checker may reject the certificate even when the original verification attempt was logically valid. This is a conservative failure mode: the framework prefers rejection over unsound acceptance. However, it also means that better producer instrumentation would improve certificate coverage.

Benchmark scale. The benchmark includes examples ranging from very small certificates to larger loop and binary-search-style certificates with thousands of proof steps. These experiments demonstrate that checking and compression are feasible for the included workload. However, larger industrial verification projects may produce substantially bigger proof traces, more complex branching structures, and richer background theories. Additional experiments are needed to evaluate scalability under such conditions.

Overall, these threats suggest that the current framework should be viewed as a conservative proof-carrying validation layer rather than a complete verified replacement for a deductive verifier. Its main contribution is to make the final verification decision depend on explicit, replayable proof evidence and a small auditable checker, while leaving proof search and automation to existing verification engines.

10 | Related Work

10.1 | Rust Verification Tools

Rust verification has attracted increasing attention because Rust combines low-level systems programming with a rich ownership and borrowing discipline. Several verification tools have therefore been developed to reason about Rust programs at different abstraction levels. Tools such as Prusti, Creusot, Verus, Kani, MIRAI, RustHorn, and RustBelt explore complementary points in the design space of Rust verification. Some approaches translate Rust or Rust-like intermediate representations into established verification backends such as Viper, Why3, Boogie, SMT solvers, or model checkers. Others provide semantic models of ownership, borrowing, lifetimes, or unsafe code, and use these models to establish soundness results for fragments of the language [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

These systems have substantially advanced the state of Rust verification, but their main goal is usually proof generation, program analysis, or semantic soundness, rather than independent validation of generated proof evidence. In a typical verifier-centered workflow, the same large verification infrastructure is responsible for parsing programs, generating verification conditions, applying proof rules, simplifying formulas, invoking automation, and deciding whether a program is verified. This design is practical and powerful, but it also makes the trusted computing base relatively large. Our work is orthogonal to these Rust verification tools. Instead of replacing an existing verifier, we treat the verifier as an untrusted proof producer and add a proof-carrying layer that emits explicit certificates and checks them with a small independent kernel. In this sense, our work is closest in spirit to source-level deductive verification for Rust, where proof rules are intended to reflect source-language constructs such as assignment, borrowing, mutable references, arrays, tuples, enums, and loop invariants. However, the contribution of this paper is not a new Rust program logic or a new proof-search strategy

10.2 | Proof-Carrying Code and Proof Certificates

Proof-carrying code introduced the idea that executable code can be accompanied by a machine-checkable proof showing that the code satisfies a required safety policy. In the original setting,

a code consumer does not need to trust the code producer; it only needs to trust a proof checker and the underlying safety policy. This idea has since influenced many systems involving certifying compilation, typed assembly language, foundational proof-carrying code, proof-producing solvers, and proof-carrying authorization. The common principle is that large or untrusted producers may construct evidence, but a smaller and more reliable checker should validate the evidence before accepting the result.

Our work follows this proof-carrying philosophy but applies it to source-level deductive verification for Rust programs. The certificate does not merely assert that verification succeeded. It records proof-step identifiers, rule names, parent-child dependencies, branch metadata, substitutions, side conditions, closure evidence, sequent snapshots or opaque sequent tokens, and replay digests. The certificate is therefore intended to be replayable and inspectable, rather than being only a proof log or a success flag.

The distinction is important for practical verification infrastructure. Many deductive verifiers produce internal proof traces, proof scripts, or solver logs, but these artifacts are often tied to the original verifier implementation. Rechecking them may require the same large toolchain that produced them. By contrast, our certificates are designed as explicit proof-carrying objects that can be checked outside the original verification engine. This makes the trust boundary clearer: the proof producer may be complex, but the final validation step depends only on the certificate format, rule-schema interface, replay discipline, and small checker [11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22].

10.3 | Small Trusted Kernels

The use of small trusted kernels is a central idea in interactive theorem proving and proof assistant design. Systems in the LCF tradition, as well as modern theorem provers such as HOL, Isabelle, Coq, and Lean, rely on a small kernel to check primitive inference steps, while automation and tactics are implemented outside the kernel. This architecture allows powerful proof search and automation to be developed without enlarging the trusted core. If a tactic is buggy, it may fail to find a proof or generate an invalid proof term, but it should not be able to convince the kernel to accept a false theorem.

Our design imports this principle into source-level program verification. Modern deductive verifiers are often not organized around a minimal proof-checking kernel. They may include front-end translations, verification-condition generation, symbolic execution, rule application, arithmetic simplification, SMT-based automation, and proof-state management inside one large trusted workflow. This is convenient for automation, but it complicates auditing because the final verification result depends on many interacting components.

The proposed checker deliberately has a much smaller responsibility. It does not perform proof search, does not invoke the original verifier, and does not attempt to reimplement the full verification engine. Instead, it validates certificate structure, known rule schemas, step dependencies, branch closure information, substitution consistency, conservative side conditions, and replay digests. This small-kernel view reduces the trusted computing base from the full verification stack to a compact certificate checker and its explicit rule-schema interface. The approach is conservative: unsupported rules, malformed certificates, invalid branch closures, inconsistent substitutions, and corrupted macro steps are rejected rather than silently accepted.

10.4 | Certificate Checking in Deductive Verification

Certificate checking has also been studied in the context of deductive program verification, SMT solving, proof-producing decision procedures, and certified verification-condition generation. For example, SMT solvers may emit proof objects that are checked by external proof checkers, and verification tools based on intermediate languages may rely on trusted transformations from programs to logical verification conditions. Systems such as Why3, Boogie, Dafny, KeY, and related deductive verifiers provide different combinations of verification condition generation, proof automation, proof obligations, and backend solver support. In many of these settings, the central challenge is to ensure that the logical evidence accepted by the verifier corresponds to a valid derivation.

Our work differs in two ways. First, the certificate is attached to source-level Rust verification evidence rather than only to backend SMT proofs or intermediate verification conditions. This means that the checker must preserve information about proof steps related to source-level constructs, including assignment, borrowing, mutable references, arrays, tuples, enum reasoning, loop-invariant rules, and branch closure. Second, the checker is designed to operate independently from the original proof producer. It validates the replay structure exposed by the certificate rather than trusting the internal state of the verification engine.

The implementation therefore supports multiple replay modes. When sufficient textual proof information is available, the checker performs schema-level textual replay for supported rule families. When the trace omits full textual sequents, the checker falls back to conservative trace-shape replay using stable opaque sequent tokens, proof-tree structure, branch metadata, closure evidence, and digests. Arithmetic simplification is handled conservatively through supported rule-family prefixes rather than by embedding a full SMT solver in the checker. This design does not claim to solve all certificate-checking problems for Rust verification, but it provides a practical and auditable replay layer for the supported proof fragment.

10.5 | Proof Compression and Replay

Proof certificates can become large, especially when verification involves symbolic execution, loop reasoning, branching proof states, or many small simplification steps. Proof compression has therefore been studied in several contexts, including resolution proofs, SMT proofs, proof-carrying code, interactive theorem proving, and proof-producing automation. Common techniques include sharing repeated subproofs, introducing macro steps, eliminating redundant inferences, compressing proof traces, and replaying compact proof objects against a trusted kernel. The main difficulty is to reduce certificate size without turning compression itself into an additional trusted component.

Our work adopts this principle for source-level verification certificates. Compressed certificates are not treated as merely smaller

files. Instead, macro steps must remain checkable under the replay discipline. The checker validates compressed proof evidence using replay metadata and digest information, and it rejects corrupted compressed macro steps. Thus, compression is not part of the trusted proof-producing process; it is an untrusted transformation whose output must still be accepted by the independent checker [23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33].

This distinction is particularly important for practical software verification. Large proof traces may be expensive to store, inspect, or transmit, but aggressive compression can obscure the relationship between the original proof and the checked evidence. Our approach balances these concerns by allowing proof fragments to be compressed while preserving enough structure for independent validation. The benchmark results show that compressed certificates can be substantially smaller than full certificates while remaining replayable by the checker. At the same time, negative tests demonstrate that corrupted rules, substitutions, side conditions, initial sequents, branch closures, and compressed macro steps are rejected. These results support the central claim of this paper: proof compression is useful only when it preserves independent checkability.

11 | Conclusion

This paper presented a proof-carrying source-level verification framework for Rust programs, with the goal of separating proof production from proof trust in deductive verification. Instead of relying on a large verification engine as the final trusted authority, the proposed approach converts proof evidence into explicit JSON certificates and validates them using a small independent checking kernel. The framework supports full and compressed certificates, replay-based checking, rule-schema validation, branch-closure validation, replay digests, and negative certificate testing. The experimental results show that the checker successfully validates certificates across representative Rust verification features, including assignment, borrowing, mutable references, tuples, arrays, enums, loop invariants, arithmetic reasoning, and binary-search-style examples. The results also demonstrate that certificate compression can substantially reduce certificate size while preserving independent checkability, and that corrupted certificates are rejected with explicit diagnostic reasons. These findings suggest that proof-carrying verification can provide a practical and auditable trust layer for source-level program verification. Although the current implementation remains conservative and supports only a bounded subset of proof-rule families, it shows that a small-kernel checker can reduce the trusted computing base without replacing the original verifier or its proof-search capabilities. Future work will extend the supported replay rules, strengthen arithmetic checking with richer certificate formats, improve source-language coverage, and explore tighter integration with existing verification backends. Overall, this work provides a practical step toward more trustworthy, inspectable, and reusable verification infrastructures for software engineering.

Author Contributions

Zichen Song designed the framework, implemented the certificate checker, conducted the experiments, and wrote the manuscript.

Weijia Li contributed to the conceptual development of the proof-carrying verification workflow, provided guidance on the experimental design and evaluation methodology, discussed the trusted-kernel architecture and certificate replay strategy, and reviewed and revised the manuscript.

Acknowledgments

The author thanks the developers and maintainers of source-level verification tools for Rust, whose work provides the foundation for further research on proof-carrying verification and independently checkable certificate systems.

Financial Disclosure

The author received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors for the research, authorship, or publication of this article.

Conflicts of Interest

The author declares that there are no conflicts of interest relevant to this work. The author has no financial, personal, professional, or institutional relationships that could be construed to have influenced the research, development, evaluation, interpretation, or presentation of the results reported in this article.

Supporting Information

No additional supporting information is available for this article. All materials necessary to understand the method, implementation, and experimental results are described in the manuscript.

Data and Code Availability

To facilitate reproducibility and independent evaluation, we make the full research artifact available online. The repository contains the certificate checker, certificate-generation utilities, representative Rust verification examples, replay-mode configurations, benchmark scripts, and negative-test certificates used to evaluate rejection behavior. It also provides example full certificates and compressed macro certificates, allowing readers to inspect how proof evidence is represented and independently replayed by the checker. These materials are provided to support transparent comparison, artifact reuse, and future development of certificate-based trust mechanisms for source-level verification systems.

References

1. Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt,, and Mattias Ulbrich. (Eds.) 2016. *Deductive Software Verification – The KeY Book: From Theory to Practice*, edited by Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt, and Mattias Ulbrich. Number 10001 in Lecture Notes in Computer Science. Springer.
2. Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle,, and Mattias Ulbrich. 2025. "The Many Uses of Dynamic Logic." In *Go Where the Bugs Are – Essays Dedicated to Wolfgang Reif on the Occasion of His 65th Birthday*, Vol 15765 of *Lecture Notes in Computer Science*, edited by G. Ernst et al., 56–82. Cham: Springer.
3. Wolfgang Ahrendt, Andreas Roth,, and Ralf Sasse. 2005. "Automatic Validation of Transformation Rules for Java Verification

against a Rewriting Semantics." In *Logic for Programming, Artificial Intelligence, and Reasoning*, edited by Geoff Sutcliffe and Andrei Voronkov, 412–426. Berlin, Heidelberg: Springer.
4. Vytautas Astrauskas et al. 2022. "The Prusti Project: Formal Verification for Rust." In *NASA Formal Methods – 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings*, edited by Jyotirmoy V. Deshmukh, Klaus Havelund, and Ivan Perez, Vol 13260 of *Lecture Notes in Computer Science*, 88–108. Springer.
5. Vytautas Astrauskas, Christoph Matheja, Federico Poli, Peter Müller,, and Alexander J. Summers. "How Do Programmers Use Unsafe Rust?." *Proceedings of the ACM on Programming Languages* 4, no. OOPSLA (2020): 136:1–136:27.
6. S. É. Ayoun, Xavier Denis, Petar Maksimović,, and Philippa Gardner. "A Hybrid Approach to Semi-Automated Rust Verification." . *arXiv preprint arXiv:2403.15122*.
7. Bernhard Beckert et al. 2025. "The Java Verification Tool KeY: A Tutorial." In *Formal Methods*, edited by André Platzer, Kristin Y. Rozier, Matteo Pradella, and Matteo Rossi, Vol 14934 of *Lecture Notes in Computer Science*, 597–623. Cham: Springer Nature Switzerland.
8. Alexandre L. Blanc, and Patrick Lam. "Surveying the Rust Verification Landscape." *CoRR*abs/2410.01981.
9. John Boyland 2013. "Fractional Permissions." In *Aliasing in Object-Oriented Programming: Types, Analysis and Verification*, Vol 7850 of *Lecture Notes in Computer Science*, edited by Dave Clarke, James Noble, and Tobias Wrigstad, 270–288. Springer.
10. Stijn de Gouw, Jurriaan Rot, Frank S. de Boer, Richard Bubel,, and Reiner Hähnle. 2015. "OpenJDK's java.utils.Collection.sort() Is Broken: The Good, the Bad and the Worst Case." In *Computer Aided Verification*, edited by Daniel Kroening and Corina S. Păsăreanu, Vol 9206 of *Lecture Notes in Computer Science*, 273–289. Cham: Springer.
11. Xavier Denis, Jacques-Henri Jourdan,, and Claude Marché. 2022. "Creusot: A Foundry for the Deductive Verification of Rust Programs." In *Formal Methods and Software Engineering*, edited by Adrián Riesco and Min Zhang, Vol 13478 of *Lecture Notes in Computer Science*, 90–105. Springer.
12. Edsger W. Dijkstra, and Carel S. Scholten. 1990. *Predicate Calculus and Program Semantics*, Monographs in Computer Science. New York, NY: Springer.
13. Crystal Chang Din, Reiner Hähnle, Ludovic Henrio, Einar Broch Johnsen, Violet Ka I Pun,, and Silvia Lizeth Tapia Tarifa. "Locally Abstract, Globally Concrete Semantics of Concurrent Programming Languages." *ACM Transactions on Programming Languages and Systems* 46, no. 1 (2024): 3:1–3:58.
14. Daniel Drodt, and Reiner Hähnle. 2026. "RustyDL: A Program Logic for Rust."
15. Jean-Christophe Filliâtre, and Andrei Paskevich. 2013. "Why3 – Where Programs Meet Provers." In *Programming Languages and Systems*, edited by Matthias Felleisen and Philippa Gardner, Vol 7792 of *Lecture Notes in Computer Science*, 125–128. Springer.
16. Reiner Hähnle. "Many-Valued Logic, Partiality, and Abstraction in Formal Specification Languages." *Logic Journal of the IGPL* 13, no. 4 (2005): 415–433.
17. Son Ho, and Jonathan Protzenko. "Aeneas: Rust Verification by Functional Translation." *Proceedings of the ACM on Programming Languages* 6, no. ICFP (2022): 711–741.
18. C. A. R. Hoare. "An Axiomatic Basis for Computer Programming." *Communications of the ACM* 12, no. 10 (1969): 576–580.
19. Marieke Huisman, and Wojciech Mostowski. 2015. "A Symbolic Approach to Permission Accounting for Concurrent Reasoning." In *14th International Symposium on Parallel and Distributed Computing, ISPDC 2015, Limassol, Cyprus, June 29–July 2, 2015*, 165–174. IEEE Computer Society.
20. Ralf Jung et al. "The Future Is Ours: Prophecy Variables in Separation Logic." *Proceedings of the ACM on Programming Languages* 4, no. POPL (2020): 45:1–45:32.
21. Steve Klabnik, and Carol Nichols. 2022. *The Rust Programming Language*, San Francisco, CA: No Starch Press.
22. Andrea Lattuada et al. "Verus: Verifying Rust Programs Using Linear Ghost Types." *Proceedings of the ACM on Programming Languages* 7, no. OOPSLA1 (2023): 286–315.
23. Gary T. Leavens et al. 2013. *JML Reference Manual*.
24. Hongyu Li, Liancheng Guo, Yao Yang, Sheng Wang,, and Ming Xu. 2024. "An Empirical Study of Rust-for-Linux: The Success, Dissatisfaction, and Compromise." In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10–12, 2024*, edited by Saurabh Bagchi and Yiying Zhang, 425–443. USENIX Association.
25. John McCarthy 1962. "Towards a Mathematical Science of Computation." In *Information Processing, Proceedings of the 2nd IFIP Congress 1962, Munich, Germany, August 27–September 1, 1962*, 21–28. North-Holland.
26. Peter Müller, Malte Schwerhoff,, and Alexander J. Summers. 2016. "Viper: A Verification Infrastructure for Permission-Based Reasoning." In *Verification, Model Checking, and Abstract Interpretation*, edited by Barbara Jobstmann and K. Rustan M. Leino, 41–62. Berlin, Heidelberg: Springer.
27. Vaughan R. Pratt 1976. "Semantical Considerations on Floyd-Hoare Logic." In *17th Annual Symposium on Foundations of Computer Science, Houston, TX, USA*, 109–121. Los Alamitos, CA: IEEE.
28. Rust Community 2024. *The Rust Reference*.
29. Rust Community 2025. *Rust Compiler Development Guide*.
30. Gerhard Schellhorn, Stefan Bodenmüller, Michael Bitterlich,, and Wolfgang Reif. 2022. "Software & System Verification with KIV." In *The Logic of Software: A Tasting Menu of Formal Methods – Essays Dedicated to Reiner Hähnle on the Occasion of His 60th Birthday*, Vol 13360 of *Lecture Notes in Computer Science*, edited by Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, and Einar Broch Johnsen, 408–436. Springer.
31. Dominic Steinhöfel, and Nathan Wasser. 2017. "A New Invariant Rule for the Analysis of Loops with Non-Standard Control Flows." In *Integrated Formal Methods – 13th International Conference, IFM 2017, Turin, Italy, September 20–22, 2017, Proceedings*, edited by Nadia Polikarpova and Steve A. Schneider, Vol 10510 of *Lecture Notes in Computer Science*, 279–294. Springer.
32. Clive Thompson. "How Rust Went from a Side Project to the World's Most-Loved Programming Language." *MIT Technology Review* 126, no. 2.
33. Karen Trentelman 2005. "Proving Correctness of JavaCard DL Taclets Using Bali." In *Proceedings of the Third IEEE International Conference on Software Engineering and Formal Methods*, SEFM '05, 160–169. USA: IEEE Computer Society.

References

1. Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt,, and Mattias Ulbrich. (Eds.) 2016. *Deductive Software Verification – The KeY Book: From Theory to Practice*, edited by Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt, and Mattias Ulbrich. Number 10001 in Lecture Notes in Computer Science. Springer.
2. Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle,, and Mattias Ulbrich. 2025. "The Many Uses of Dynamic Logic." In *Go Where the Bugs Are – Essays Dedicated to Wolfgang Reif on the Occasion of His 65th Birthday*, Vol 15765 of *Lecture Notes in Computer Science*, edited by G. Ernst et al., 56–82. Cham: Springer.
3. Wolfgang Ahrendt, Andreas Roth,, and Ralf Sasse. 2005. "Automatic Validation of Transformation Rules for Java Verification against a Rewriting Semantics." In *Logic for Programming, Artificial Intelligence, and Reasoning*, edited by Geoff Sutcliffe and Andrei Voronkov, 412–426. Berlin, Heidelberg: Springer.

4. Vytautas Astrauskas et al. 2022. "The Prusti Project: Formal Verification for Rust." In *NASA Formal Methods – 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings*, edited by Jyotirmoy V. Deshmukh, Klaus Havelund, and Ivan Perez, Vol 13260 of *Lecture Notes in Computer Science*, 88–108. Springer.
5. Vytautas Astrauskas, Christoph Matheja, Federico Poli, Peter Müller,, and Alexander J. Summers. "How Do Programmers Use Unsafe Rust?." *Proceedings of the ACM on Programming Languages* 4, no. OOPSLA (2020): 136:1–136:27.
6. S. É. Ayoun, Xavier Denis, Petar Maksimović,, and Philippa Gardner. "A Hybrid Approach to Semi-Automated Rust Verification." . *arXiv preprint arXiv:2403.15122*.
7. Bernhard Beckert et al. 2025. "The Java Verification Tool KeY: A Tutorial." In *Formal Methods*, edited by André Platzer, Kristin Y. Rozier, Matteo Pradella, and Matteo Rossi, Vol 14934 of *Lecture Notes in Computer Science*, 597–623. Cham: Springer Nature Switzerland.
8. Alexandre L. Blanc, and Patrick Lam. "Surveying the Rust Verification Landscape." *CoRR*abs/2410.01981.
9. John Boyland 2013. "Fractional Permissions." In *Aliasing in Object-Oriented Programming: Types, Analysis and Verification*, Vol 7850 of *Lecture Notes in Computer Science*, edited by Dave Clarke, James Noble, and Tobias Wrigstad, 270–288. Springer.
10. Stijn de Gouw, Jurriaan Rot, Frank S. de Boer, Richard Bubel,, and Reiner Hähnle. 2015. "OpenJDK's java.utils.Collection.sort() Is Broken: The Good, the Bad and the Worst Case." In *Computer Aided Verification*, edited by Daniel Kroening and Corina S. Păsăreanu, Vol 9206 of *Lecture Notes in Computer Science*, 273–289. Cham: Springer.
11. Xavier Denis, Jacques-Henri Jourdan,, and Claude Marché. 2022. "Creusot: A Foundry for the Deductive Verification of Rust Programs." In *Formal Methods and Software Engineering*, edited by Adrián Riesco and Min Zhang, Vol 13478 of *Lecture Notes in Computer Science*, 90–105. Springer.
12. Edsger W. Dijkstra, and Carel S. Scholten. 1990. *Predicate Calculus and Program Semantics*, Monographs in Computer Science. New York, NY: Springer.
13. Crystal Chang Din, Reiner Hähnle, Ludovic Henrio, Einar Broch Johnsen, Violet Ka I Pun,, and Silvia Lizeth Tapia Tarifa. "Locally Abstract, Globally Concrete Semantics of Concurrent Programming Languages." *ACM Transactions on Programming Languages and Systems* 46, no. 1 (2024): 3:1–3:58.
14. Daniel Drodt, and Reiner Hähnle. 2026. "RustyDL: A Program Logic for Rust."
15. Jean-Christophe Filliâtre, and Andrei Paskevich. 2013. "Why3 – Where Programs Meet Provers." In *Programming Languages and Systems*, edited by Matthias Felleisen and Philippa Gardner, Vol 7792 of *Lecture Notes in Computer Science*, 125–128. Springer.
16. Reiner Hähnle. "Many-Valued Logic, Partiality, and Abstraction in Formal Specification Languages." *Logic Journal of the IGPL* 13, no. 4 (2005): 415–433.
17. Son Ho, and Jonathan Protzenko. "Aeneas: Rust Verification by Functional Translation." *Proceedings of the ACM on Programming Languages* 6, no. ICFP (2022): 711–741.
18. C. A. R. Hoare. "An Axiomatic Basis for Computer Programming." *Communications of the ACM* 12, no. 10 (1969): 576–580.
19. Marieke Huisman, and Wojciech Mostowski. 2015. "A Symbolic Approach to Permission Accounting for Concurrent Reasoning." In *14th International Symposium on Parallel and Distributed Computing, ISPDC 2015, Limassol, Cyprus, June 29–July 2, 2015*, 165–174. IEEE Computer Society.
20. Ralf Jung et al. "The Future Is Ours: Prophecy Variables in Separation Logic." *Proceedings of the ACM on Programming Languages* 4, no. POPL (2020): 45:1–45:32.
21. Steve Klabnik, and Carol Nichols. 2022. *The Rust Programming Language*, San Francisco, CA: No Starch Press.
22. Andrea Lattuada et al. "Verus: Verifying Rust Programs Using Linear Ghost Types." *Proceedings of the ACM on Programming Languages* 7, no. OOPSLA1 (2023): 286–315.
23. Gary T. Leavens et al. 2013. *JML Reference Manual*.
24. Hongyu Li, Liancheng Guo, Yao Yang, Sheng Wang,, and Ming Xu. 2024. "An Empirical Study of Rust-for-Linux: The Success, Dissatisfaction, and Compromise." In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10–12, 2024*, edited by Saurabh Bagchi and Yiying Zhang, 425–443. USENIX Association.
25. John McCarthy 1962. "Towards a Mathematical Science of Computation." In *Information Processing, Proceedings of the 2nd IFIP Congress 1962, Munich, Germany, August 27–September 1, 1962*, 21–28. North-Holland.
26. Peter Müller, Malte Schwerhoff,, and Alexander J. Summers. 2016. "Viper: A Verification Infrastructure for Permission-Based Reasoning." In *Verification, Model Checking, and Abstract Interpretation*, edited by Barbara Jobstmann and K. Rustan M. Leino, 41–62. Berlin, Heidelberg: Springer.
27. Vaughan R. Pratt 1976. "Semantical Considerations on Floyd-Hoare Logic." In *17th Annual Symposium on Foundations of Computer Science, Houston, TX, USA*, 109–121. Los Alamitos, CA: IEEE.
28. Rust Community 2024. *The Rust Reference*.
29. Rust Community 2025. *Rust Compiler Development Guide*.
30. Gerhard Schellhorn, Stefan Bodenmüller, Michael Bitterlich,, and Wolfgang Reif. 2022. "Software & System Verification with KIV." In *The Logic of Software: A Tasting Menu of Formal Methods – Essays Dedicated to Reiner Hähnle on the Occasion of His 60th Birthday*, Vol 13360 of *Lecture Notes in Computer Science*, edited by Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, and Einar Broch Johnsen, 408–436. Springer.
31. Dominic Steinhöfel, and Nathan Wasser. 2017. "A New Invariant Rule for the Analysis of Loops with Non-Standard Control Flows." In *Integrated Formal Methods – 13th International Conference, IFM 2017, Turin, Italy, September 20–22, 2017, Proceedings*, edited by Nadia Polikarpova and Steve A. Schneider, Vol 10510 of *Lecture Notes in Computer Science*, 279–294. Springer.
32. Clive Thompson. "How Rust Went from a Side Project to the World's Most-Loved Programming Language." *MIT Technology Review* 126, no. 2.
33. Karen Trentelman 2005. "Proving Correctness of JavaCard DL Taclets Using Bali." In *Proceedings of the Third IEEE International Conference on Software Engineering and Formal Methods*, SEFM '05, 160–169. USA: IEEE Computer Society.